

✓ Implementing RLHF with PPO study

I take this homework as opportunity to explore RLHF in a hour. Long term for short, Reinforcement Learning with Human Feedback (RLHF) is a popular approach in the field of natural language processing that aims to optimize language models for human preferences directly, rather than solely relying on traditional training methods such as supervised or unsupervised learning.

Training Approach for RLHF (source):

Collect human feedback

Train a reward model Optimize LLM against the reward model

I will write my understanding toward to the RLHF/PPO part

```
#!git clone https://github.com/CarperAI/trlx.git
!git config --global --add safe.directory /content/trlx && cd /content/trlx && pip install -e .

# uninstall scikit_learn + jax to avoid numpy issues
!pip uninstall -y scikit_learn jax

import os

# run within repo
os.chdir('/content/trlx/examples/summarize_rlhf/')
print(os.getcwd())

!pip install -r requirements.txt
!pip install mpi4py

# run within reward model directory
os.chdir('/content/trlx/examples/summarize_rlhf/reward_model/')
print(os.getcwd())
```

↔ 显示隐藏的输出项

✓ Generate dataset

```
from transformers import pipeline, set_seed
import json

def generate_examples(prompt_list, model_name='gpt2', max_length=50, num_return_sequences=2, seed=42):
    generator = pipeline('text-generation', model=model_name, device=0)
    set_seed(seed)
    examples = []
    for prompt in prompt_list:
        result = generator(prompt, max_length=max_length, num_return_sequences=num_return_sequences)
        example = {'prompt': prompt}
        for i, res in enumerate(result):
            answer = res['generated_text'].lstrip().removeprefix(prompt).strip()
            example[f'answer{i + 1}'] = answer
        examples.append(example)
        print(json.dumps(example, indent=2))
    return examples

prompts = [
    "What is the latest news on the stock market?",
```

```

    "What is the current state of the economy?",
    "What are the latest developments in technology?",
    "What is the political situation in the Middle East?",
    "What are the latest trends in fashion and beauty?",
    "What are the top travel destinations for this year?",
    "What are some healthy recipes for a vegan diet?",
    "What are the most important events happening in the world today?",
    "What are some tips for improving mental health?",
    "What are the best ways to save money for retirement?",
    "What are some popular new books or movies?",
    "What are some effective ways to reduce stress?",
    "What are the latest developments in artificial intelligence?",
    "What are some top-rated restaurants in your city?",
    "What are the best ways to stay fit and healthy?",
    "What are some tips for successful entrepreneurship?",
    "What are some effective ways to improve productivity?",
    "What are the latest developments in climate change research?",
    "What are some top-rated TV shows or movies on streaming services?",
    "What are some fun activities to do on weekends?",
    "What are some effective ways to manage time and prioritize tasks?",
    "What are the latest trends in home decor and design?",
    "What are the best ways to develop a successful career?",
    "What are some popular new products or gadgets?",
    "What are some effective ways to improve communication skills?",
    "What are some tips for successful relationships?",
    "What are the latest developments in space exploration?",
    "What are some top-rated online courses or certifications?",
    "What are some effective ways to improve public speaking skills?",
    "What are the latest trends in digital marketing?",
    "What are some fun and creative DIY projects?",
    "What are some effective ways to improve leadership skills?"
]

```

```
generated_examples = generate_examples(prompts)
```

```

# Save generated examples to import in Label Studio
with open('ls_input_data.json', 'w') as f:
    json.dump(generated_examples, f, indent=2)

```

 显示隐藏的输出项

▼ human labeling

The code that those generated answer and use label studio for labeling to get human answer perferance! Bascally labeling about user perfer answer 1 or answer 2

Then we would train a preference model with our custom dataset

I will skip this part and assume we already did the labeling

```
# 此内容为代码格式
```

```
import codecs
```

```

# This file is generated by Label Studio after completing annotations
data_path = '/content/ls_output_data.json'

```

```
with codecs.open(data_path, 'r', encoding='utf-8') as f:
    data = json.load(f)
print(data)
```

```
➞ [{"id": 401635, "annotations": [{"id": 68045, "completed_by": 2, "result": [{"value": {"selected": "left"}, "id": "Sba4F4tuVh", "from_name": "comparison", "to_name": "comparison", "type": "pairwise", "origin": "manual"}], "was_cance
```

开始借助 AI 编写或生成代码。

✓ trained a reward model based on labeled that

The reward model is a GPT model that bascially do Pairwise claccification that tell choose or reject which one based on human preference on labeled data

```
import os

import torch
from datasets import load_dataset
from reward_model import GPTRewardModel
from torch.utils.data import Dataset
from tqdm import tqdm
from transformers import AutoTokenizer, Trainer, TrainingArguments

def create_comparison_dataset_ls(path: str):
    with codecs.open(data_path, 'r', encoding='utf-8') as f:
        data = json.load(f)
    pairs = []
    for sample in data:
        chosen = None
        rejected = None
        for annotation in sample['annotations']:
            if annotation['result'][0]['value']['selected'] == 'left':
                chosen = sample['data']['prompt'] + '\n' + sample['data']['answer1']
                rejected = sample['data']['prompt'] + '\n' + sample['data']['answer2']
            else:
                chosen = sample['data']['prompt'] + '\n' + sample['data']['answer2']
                rejected = sample['data']['prompt'] + '\n' + sample['data']['answer1']
        pair = {
            'chosen': chosen,
            'rejected': rejected
        }
        pairs.append(pair)

    return pairs

class PairwiseDataset(Dataset):
    def __init__(self, pairs, tokenizer, max_length):
        self.chosen_input_ids = []
        self.chosen_attn_masks = []
        self.rejected_input_ids = []
        self.rejected_attn_masks = []
        for pair in tqdm(pairs):
            chosen, rejected = pair["chosen"], pair["rejected"]
            chosen_encodings_dict = tokenizer(
                "<|startoftext|>" + chosen + "<|endoftext|>",
                truncation=True,
                max_length=max_length,
                padding="max_length",
```

```

        return_tensors="pt",
    )
    rejected_encodings_dict = tokenizer(
        "<|startoftext|>" + rejected + "<|endoftext|>",
        truncation=True,
        max_length=max_length,
        padding="max_length",
        return_tensors="pt",
    )
    self.chosen_input_ids.append(chosen_encodings_dict["input_ids"])
    self.chosen_attn_masks.append(chosen_encodings_dict["attention_mask"])
    self.rejected_input_ids.append(rejected_encodings_dict["input_ids"])
    self.rejected_attn_masks.append(rejected_encodings_dict["attention_mask"])

def __len__(self):
    return len(self.chosen_input_ids)

def __getitem__(self, idx):
    return (
        self.chosen_input_ids[idx],
        self.chosen_attn_masks[idx],
        self.rejected_input_ids[idx],
        self.rejected_attn_masks[idx],
    )

class DataCollatorReward:
    def __call__(self, data):
        batch = {}
        batch["input_ids"] = torch.cat([f[0] for f in data] + [f[2] for f in data])
        batch["attention_mask"] = torch.cat([f[1] for f in data] + [f[3] for f in data])
        batch["labels"] = torch.tensor([0] * len(data) + [1] * len(data))
        return batch

def compute_metrics(eval_preds):
    chosen_end_scores = eval_preds.predictions[0] # chosen scores
    rejected_end_scores = eval_preds.predictions[1] # rejected scores

    result = {}
    acc = sum(chosen_end_scores > rejected_end_scores) / len(rejected_end_scores)
    result["accuracy"] = acc

    return result

```

双击 (或按回车键) 即可修改

```

tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token

if not os.path.exists("rm_checkpoint"):
    os.mkdir("rm_checkpoint")

# Initialize the reward model from the GPT-2 model (optionally SFT GPT-2)
model = GPTRewardModel("gpt2")

# Freeze the first 70% of the hidden layers of the reward model backbone
layers = model.transformer.h

```

```

num_layers = len(layers)
num_unfrozen = int(0.3 * num_layers)
for layer in layers[:num_unfrozen]:
    layer.requires_grad_(False)

# Create the comparisons datasets
pairs = create_comparison_dataset_ls(data_path)
train_size = int(0.8 * len(pairs)) # 80% training, 20% validation
train_pairs = pairs[0:train_size]
val_pairs = pairs[train_size:]

# Make pairwise datasets for training
max_length = 550
train_dataset = PairwiseDataset(train_pairs, tokenizer, max_length=max_length)
val_dataset = PairwiseDataset(val_pairs, tokenizer, max_length=max_length)

# Create the collator to gather batches of pairwise comparisons
data_collator = DataCollatorReward()

```



```

tokenizer_config.json: 100% 619/619 [00:00<00:00, 48.5kB/s]
vocab.json: 100% 798k/798k [00:00<00:00, 4.09MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 3.45MB/s]
tokenizer.json: 100% 1.37M/1.37M [00:00<00:00, 6.80MB/s]
added_tokens.json: 100% 4.04k/4.04k [00:00<00:00, 309kB/s]
special_tokens_map.json: 100% 357/357 [00:00<00:00, 29.7kB/s]

```

```
100%|██████████| 25/25 [00:00<00:00, 929.63it/s]
```

```

training_args = TrainingArguments(
    output_dir="rm_checkpoint/",
    num_train_epochs=50,
    logging_steps=10,
    gradient_accumulation_steps=4,
    save_strategy="steps",
    evaluation_strategy="steps",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    eval_accumulation_steps=1,
    eval_steps=10,
    save_steps=10,
    warmup_steps=100,
    logging_dir="./logs",
    fp16=True,
    bf16=False,
    learning_rate=1e-5,
    # deepspeed="ds_config_gpt_j.json",
    save_total_limit=1
)

```

```

Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    compute_metrics=compute_metrics,
    eval_dataset=val_dataset,

```

```
data_collator=data_collator,
).train()
```

```

/usr/local/lib/python3.10/dist-packages/transformers/training_args.py:1525: FutureWarning: `evaluation_strategy` is deprecated and will be removed in version 4.46 of 🤗 Transformers. Use `eval_strategy` instead
warnings.warn(
/usr/local/lib/python3.10/dist-packages/accelerate/accelerator.py:494: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
self.scaler = torch.cuda.amp.GradScaler(**kwargs)
[2024-10-28 01:27:13,588] [INFO] [real_accelerator.py:219:get_accelerator] Setting ds_accelerator to cuda (auto detect)
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not intended, please specify a different run name by setting the `TrainingArguments.run_name` param
wandb: Using wandb-core as the SDK backend. Please refer to https://wandb.me/wandb-core for more information.
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: .....
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: .....
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: .....
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
Tracking run with wandb version 0.18.5
Run data is saved locally in /content/trlx/examples/summarize_rlhf/reward_model/wandb/run-20241028_012804-s4457x3f
Syncing run rm\_checkpoint/ to Weights & Biases \(docs\)
View project at https://wandb.ai/gbh5146-penn-state/huggingface
View run at https://wandb.ai/gbh5146-penn-state/huggingface/runs/s4457x3f
/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:825: UserWarning: cuDNN SDPA backward got grad_output.strides() != output.strides(), attempting to materialize a grad_output with matching strides.
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
[50/50 00:52, Epoch 28/50]
```

Step	Training Loss	Validation Loss	Accuracy
10	1.121900	1.016236	0.285714
20	1.110900	1.012301	0.285714
30	1.086200	1.002968	0.285714
40	1.076600	0.989344	0.285714
50	1.022300	0.969579	0.285714

```

/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:825: UserWarning: cuDNN SDPA backward got grad_output.strides() != output.strides(), attempting to materialize a grad_output with matching strides.
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:825: UserWarning: cuDNN SDPA backward got grad_output.strides() != output.strides(), attempting to materialize a grad_output with matching strides.
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:825: UserWarning: cuDNN SDPA backward got grad_output.strides() != output.strides(), attempting to materialize a grad_output with matching strides.
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
/usr/local/lib/python3.10/dist-packages/torch/autograd/graph.py:825: UserWarning: cuDNN SDPA backward got grad_output.strides() != output.strides(), attempting to materialize a grad_output with matching strides.
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
TrainOutput(global_step=50, training_loss=1.0835860443115235, metrics={'train_runtime': 107.6849, 'train_samples_per_second': 11.608, 'train_steps_per_second': 0.464, 'total_flos': 0.0, 'train_loss':
1.0835860443115235, 'epoch': 28.571428571428573})
```

```
# chang to summarize example directory
os.chdir('/content/trlx/examples/summarize_rlhf/')
print(os.getcwd())
```

```
/content/trlx/examples/summarize_rlhf
```

```

import os
from typing import List

import torch
from datasets import load_dataset
from reward_model import GPTRewardModel
from tqdm import tqdm
from transformers import AutoTokenizer
print(os.getcwd())
import trlx
from trlx.data.configs import (
    ModelConfig,
    OptimizerConfig,
```

```

    SchedulerConfig,
    TokenizerConfig,
    TrainConfig,
    TRLConfig,
)
from trlx.models.modeling_ppo import PPOConfig

↗ /content/trlx/examples/summarize_rlhf
ModuleNotFoundError: Traceback (most recent call last)
<ipython-input-26-b3ffd6cc5a99> in <cell line: 11>()
      9 print(os.getcwd())
     10 import trlx
--> 11 from trlx.data.configs import (
      12     ModelConfig,
      13     OptimizerConfig,

ModuleNotFoundError: No module named 'trlx.data'

-----
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
-----

```

After this step, it would be using the reward function to train the language model by PPO. However, the notebook cannot support colab enviroment so I guess I will start to write comment and explain what they want to do

```

# Path for reward model checkpoint: for my previous trained
REWARD_CHECKPOINT_PATH = "reward_model/rm_checkpoint/checkpoint-50/pytorch_model.bin"
SFT_MODEL_PATH = "gpt2"
# Configure TRL (Transformer Reinforcement Learning) parameters

```

```

config = TRLConfig(
    train=TrainConfig(
        seq_length=550,
        epochs=50,
        total_steps=100000,
        batch_size=4,
        checkpoint_interval=10000,
        eval_interval=200,
        pipeline="PromptPipeline",
        trainer="AcceleratePPOTrainer",
    ),
    model=ModelConfig(
        model_path="gpt2",
        num_layers_unfrozen=8,
    ),
    tokenizer=TokenizerConfig(
        tokenizer_path="gpt2",
        truncation_side="right",
    ),
    optimizer=OptimizerConfig(
        name="adamw",
        kwargs={
            "lr": 5.0e-6,
            "betas": [0.9, 0.999],
            "eps": 1.0e-8,

```

```

        "weight_decay": 0.01,
    },
),
scheduler=SchedulerConfig(
    name="cosine_annealing",
    kwargs={
        "T_max": 100000,
        "eta_min": 5.0e-6,
    },
),
method=PPOConfig(
    name="PPOConfig",
    num_rollouts=128, # Number of rollouts (trajectories) to collect per iteration
    chunk_size=16, # Number of experiences to process per chunk during training
    ppo_epochs=4, # Number of epochs to train the PPO policy for each iteration
    init_kl_coef=0.1, # Initial value of the KL coefficient for adaptive KL control
    target=6, # Target KL divergence for adaptive KL control
    horizon=10000, # Horizon (number of steps) for advantage estimation
    gamma=1, # Discount factor for future rewards
    lam=0.95, # Lambda for generalized advantage estimation (GAE)
    cliprange=0.2, # Clipping range for the policy ratio in the PPO loss
    cliprange_value=0.2, # Clipping range for the value function in the PPO loss
    vf_coef=0.2, # Coefficient for the value function loss
    scale_reward=None, # Scaling factor for the reward (if needed)
    ref_mean=None, # Reference mean for reward normalization (if needed)
    ref_std=None, # Reference standard deviation for reward normalization (if needed)
    cliprange_reward=10, # Clipping range for the reward (if needed)
    gen_kwargs={
        "max_new_tokens": 50, # Maximum number of new tokens to generate during rollouts
    },
),
)

# Load the pre-trained reward model
rw_tokenizer = AutoTokenizer.from_pretrained("gpt2")
rw_tokenizer.pad_token = rw_tokenizer.eos_token
rw_model = GPTRewardModel(SFT_MODEL_PATH)
rw_model.load_state_dict(torch.load(REWARD_CHECKPOINT_PATH))
rw_model.half()
rw_model.eval()
rw_device = torch.device("cuda:{}".format(1)) # set reward model device
rw_model.to(rw_device)

def get_scores(samples: List[str]):
    scores_list = []
    batch_size = 2
    for i in range(0, len(samples), batch_size):
        sub_samples = samples[i : i + batch_size]
        sub_samples = ["<|startoftext|>" + chosen + "<|endoftext|>" for chosen in sub_samples]
        encodings_dict = rw_tokenizer(
            sub_samples,
            truncation=True,
            max_length=config.train.seq_length,
            padding="max_length",
            return_tensors="pt",
        )
        input_ids = encodings_dict["input_ids"].to(rw_device)
        attn_masks = encodings_dict["attention_mask"].to(rw_device)
        input_ids = input_ids.repeat(2, 1)

```



```

        attn_masks = attn_masks.repeat(2, 1)
        with torch.no_grad():
            sub_scores = rw_model(input_ids=input_ids, attention_mask=attn_masks)
            scores_list.append(sub_scores["chosen_end_scores"])
    scores = torch.cat(scores_list, dim=0)
    return scores

def get_prompt_dataset(prompts, max_length):
    """
    Get the prompt after T5 decoding to make sure dictionary
    of prompts and summaries is consistent decode prompt from trlx pipeline
    """
    formatted_prompts = []
    for i in tqdm(range(len(prompts))):
        tmp = tokenizer.decode(
            tokenizer(
                prompts[i].split("TL;DR:") [0],
                truncation=True,
                max_length=max_length - 5, # to make sure "TL;DR" dont get truncated
                add_special_tokens=False,
            )["input_ids"],
            skip_special_tokens=True,
        ).strip()
        tmp = tmp + "\nTL;DR:"
        tmp = tokenizer.decode(
            tokenizer(tmp, truncation=True, max_length=max_length, add_special_tokens=False)["input_ids"],
            skip_special_tokens=True,
        ).strip()
        formatted_prompts.append(tmp)
    return formatted_prompts

def reward_fn(samples: List[str], **kwargs):
    original_samples = [text.split("TL;DR:") [0] + "TL;DR: " for text in samples]
    original_samples = [text + post_summary_dict[text.strip()] for text in original_samples]
    original_scores = get_scores(original_samples)
    scores = get_scores(samples)
    norms_scores = scores - original_scores
    return norms_scores

```

For code above, it bascially take our trained reward model: a GPT2 reward based on the Human labeling result, It will give a reward score based on given prompt. E.g. reward_fn and get_score. so we can see this training is bascially as with the reward model getting the score. then based on that, it will influence the training result.

How PPO is employed:

Internally, the trlx.train function, along with the PPO configuration, performs the following key steps:

Data Collection: The model interacts with the environment (prompts) and generates responses. These interactions are stored as trajectories or rollouts.

Reward Calculation: The reward_fn is applied to evaluate the quality of the generated responses, providing reward signals.

Policy Update: Using the collected data and rewards, the PPO algorithm updates the model's parameters to improve its text generation capabilities. It strives to maximize the expected reward over time.

By combining the reward model, training prompts, evaluation data, and the PPO configuration within trlx.train, you are essentially setting up a system to fine-tune a language model to generate more human-preferred text through reinforcement learning. This approach enables the model

to learn from feedback and iteratively refine its performance.

```
tokenizer = AutoTokenizer.from_pretrained(config.tokenizer.tokenizer_path)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "left"
max_length_input = config.train.seq_length - config.method.gen_kwargs["max_new_tokens"]

dataset = load_dataset("CarperAI/openai_summarize_tldr")

# Store data into prompt and label pairs
train_set = [(sample["prompt"], sample["label"]) for sample in dataset["train"]]
val_set = [(sample["prompt"], sample["label"]) for sample in dataset["valid"]]

# Split contents into summaries and labels
train_posts, train_summaries = zip(*train_set)
val_posts, val_summaries = zip(*val_set)

# Get the OpenAI summaries
post_summary_dict = {}
train_prompts = get_prompt_dataset(train_posts, max_length_input)
for i in range(len(train_prompts)):
    post_summary_dict[train_prompts[i]] = train_summaries[i]
val_prompts = get_prompt_dataset(val_posts, max_length_input)
for i in range(len(val_prompts)):
    post_summary_dict[val_prompts[i]] = val_summaries[i]

trainer = trlx.train(
    reward_fn=reward_fn,
    prompts=train_prompts,
    eval_prompts=val_prompts[0:1000], # sampling 1000 validation prompts for evaluation speed in training
    config=config,
)
```

双击（或按回车键）即可修改